

DBMS & SQL Mastery Study Guide

Gate Smashers Notes: Commands, Operations, Constraints & Subqueries

1. SQL Command Categorization (DDL vs DML)

Core Taxonomy

DDL (Data Definition Language)

Structure

Defines, alters, and manages database schema structures.

- **CREATE:** Builds a new table or schema.
- **ALTER:** Modifies existing table structure.
- **DROP:** Permanently removes table & structure.
- **TRUNCATE:** Removes all rows; retains structure.

DML (Data Manipulation Language)

Data Rows

Manages and manipulates actual data records inside tables.

- **INSERT:** Adds new data rows.
- **UPDATE:** Modifies existing row values.
- **DELETE:** Removes specified rows (supports WHERE).

2. Structural & Data Operations

Syntax & Comparison

CREATE TABLE Syntax

```
CREATE TABLE employee ( id INT, name  
VARCHAR(20), address VARCHAR(30) );
```

ALTER vs UPDATE

```
-- ALTER: Changes Table Schema (DDL)  
ALTER TABLE employee ADD salary INT; --  
UPDATE: Changes Data Rows (DML) UPDATE  
employee SET salary = salary * 2 WHERE id =  
1;
```

The Data Removal Matrix: DELETE vs TRUNCATE vs DROP

Command	Type	WHERE Clause?	Action & Structure Impact
DELETE	DML	Yes	Deletes specified rows. Table structure and remaining data stay intact.
TRUNCATE	DDL	No	Deletes ALL rows instantly. Table structure is preserved.
DROP	DDL	No	Destroys the entire table (data + schema completely removed).

3. SQL Integrity Constraints

Data Validation

Constraint	Core Purpose & Behavior Rule
PRIMARY KEY	Uniquely identifies each record. Automatically enforces UNIQUE + NOT NULL .
FOREIGN KEY	Maintains referential integrity between parent and child tables.
UNIQUE	Ensures all values in a column are distinct (prevents duplicates).
NOT NULL	Ensures that a column cannot store empty or NULL values.
CHECK	Restricts the value range placed in a column according to a logical condition.
DEFAULT	Provides a predefined fallback value when no explicit value is provided during INSERT.

4. Advanced Subquery Patterns

Nested Logic

💡 Subquery Fundamentals

A **subquery** is a query nested inside another query. The *inner query* executes first, and its result is evaluated by the *outer query*.

1. Find Maximum Salary

```
SELECT MAX(salary) FROM employee;
```

2. Find Employee with Maximum Salary

```
SELECT ename FROM employee WHERE  
salary = ( SELECT MAX(salary) FROM employee  
);
```

3. Find Second-Highest Salary

```
SELECT MAX(salary) FROM employee  
WHERE salary != ( SELECT MAX(salary) FROM  
employee );
```

4. Employee with Second-Highest Salary

```
SELECT ename FROM employee WHERE  
salary = ( SELECT MAX(salary) FROM employee  
WHERE salary != ( SELECT MAX(salary) FROM  
employee ) );
```

5. Aggregation with GROUP BY Pattern

Data Summarization

The **GROUP BY** clause groups rows that share identical values in specified columns into summary rows. Aggregate functions (COUNT, SUM, AVG, MAX, MIN) then operate on each isolated group.

Query: Count Employees per Department

```
SELECT department, COUNT(*) FROM  
employee GROUP BY department;
```

Execution Visual Example

Department	COUNT(*)
HR	2
Marketing	2
IT	1

Key Idea: Aggregate functions process individual groups formed by **GROUP BY**, rather than evaluating the whole table at once.

6. Exam Revision Summary

Quick Recall

DDL vs DML

DDL = Schema & Structure.
DML = Data Rows & Tuples.

Deletion Rule

DELETE = Selective rows.
TRUNCATE = All rows.
DROP = Table schema.

Subquery Logic

Nth Highest = Filter top (N-1) max values, then find MAX.

7. Common Table Expression (CTE) / WITH Clause

Advanced SQL



What is a CTE?

A **Common Table Expression (CTE)**, defined using the **WITH** keyword, allows you to create a temporary named result set that can be referenced within a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement. **It is NOT permanently stored in the database.**

Basic Syntax

```
WITH T1 AS ( SELECT column_name FROM
table_name ) SELECT * FROM T1;
```

Key Vocabulary

- **WITH** → Initiates the CTE block.
- **T1** → Temporary table alias name.
- **(...)** → Subquery generating temporary data.
- **Main Query** → References T1 directly.

8. CTE Practical Examples

Applications

Example 1: Find Employee with Max Salary

```
WITH T1 AS ( SELECT MAX(salary) AS x
FROM employee ) SELECT id, name FROM
employee WHERE salary = (SELECT x FROM T1);
```

Example 2: 2nd Highest Salary via CTE

```
WITH T1 AS ( SELECT id, salary FROM
employee WHERE salary < ( SELECT MAX(salary)
FROM employee ) ) SELECT id FROM T1 WHERE
salary = ( SELECT MAX(salary) FROM T1 );
```

9. CTE Execution Workflow & Subquery Comparison

Concepts



Second Highest Salary Pipeline via CTE

Original Table → CTE Excludes Highest Salary (95,000) → Temporary Table T1 Created → Find MAX(salary) from T1 (90,000) → Fetch Matching Employee ID (2)

Comparison Matrix: CTE vs Traditional Subquery

Feature / Aspect	Common Table Expression (CTE)	Standard Subquery
Syntax Keyword	Declared with WITH T1 AS (...) at top.	Embedded directly within WHERE or FROM clauses.
Readability	High readability; modularizes complex queries.	Can become deeply nested and difficult to maintain.
Reusability	Can be referenced multiple times within main query.	Must be re-written or re-computed if needed twice.
Persistence	Temporary for query scope (not saved on disk).	Temporary for query scope (not saved on disk).



One-Line Exam Revision

CTE (WITH) = Give a readable name to a temporary query result set and use it in your main query.

10. Filtering Groups: The HAVING Clause

Group Filtering



Core Concept: Filtering Groups vs. Filtering Rows

The HAVING clause is used to filter **groups created by GROUP BY**, typically based on aggregate condition checks (e.g., COUNT(*), SUM(), AVG()). Standard row filters (WHERE) cannot operate on aggregate values because row filtering happens *before* grouping occurs.

Key Distinction Matrix: WHERE vs. GROUP BY vs. HAVING

Clause	Primary Function	Execution Point & Rule
WHERE	Filters individual tuples/rows	Executes before grouping; cannot evaluate aggregate functions.
GROUP BY	Groups rows sharing column values	Forms summary groups for aggregate operations.
HAVING	Filters summary groups	Executes after grouping; evaluates conditions on aggregates.

11. Practical Scenario: Departments with < 2 Employees

Worked Example

Problem 1: Find Department Names

Display department names where the employee count is strictly less than 2.

Execution Pipeline

Department	COUNT(*)	HAVING < 2
HR	2	FALSE
Marketing	2	FALSE
IT	1	TRUE (Output: IT)

```
SELECT department FROM employee
GROUP BY department HAVING COUNT(*) < 2;
```

12. Combining HAVING with Outer Subqueries

Advanced Pattern

Problem 2: Find Employee Names – Display the names of employees who belong to any department that has fewer than 2 employees.

```
-- Outer Query: Retrieves matching employee names
SELECT ename FROM employee WHERE
department IN ( -- Inner Query: Filters departments using GROUP BY + HAVING
SELECT department
FROM employee GROUP BY department HAVING COUNT(*) < 2 );
```



Essential Exam Pattern Template

```
SELECT column FROM table GROUP BY column HAVING aggregate_function(condition);
```

WHERE

Row-level filter

GROUP BY

Group creation

HAVING

Group-level filter

Department-Wise Highest Salary

Using GROUP BY, MAX(), and Multi-Value Subqueries with IN

13. Finding Department-Wise Highest Salary

Concept & Workflow

Question Requirement: Find the highest salary in each department and display the employee name who earns that salary.

Step 1: Find Highest Salary Department-Wise

```
SELECT department, MAX(salary) FROM
employee GROUP BY department;
```

Returns multiple max values: 30000, 40000, 50000

Sample Intermediate Output

Department	Highest Salary
HR	30,000
Marketing	40,000
IT	50,000

14. Step 2: Retrieve Employee Names Using IN

Subquery Filtering

```
SELECT ename FROM employee WHERE salary IN ( SELECT MAX(salary) FROM employee GROUP BY
department );
```

Why use IN instead of = ?

Operator Rule

- **= Operator:** Used only when subquery returns a *single exact value* (e.g., Salary = 50000).
- **IN Operator:** Required when subquery returns *multiple values* (e.g., (30000, 40000, 50000)).

Query Output (Gate Smashers Example)

Result

ename	Department Earned
Ravi	HR (30,000)
Nitin	Marketing (40,000)
Varun	IT (50,000)

15. Exam Pattern & Important Limitation Warning

Exam Tip

Pattern To Remember

Department-wise calculation → GROUP BY

Highest value → MAX()

Multiple subquery results → IN (. . .)

Important Limitation of this Exact Query

This query matches any employee whose salary equals *any* department's maximum. If two different departments share the same maximum salary (e.g., HR max = 40k and Marketing max = 40k), it may return false-positive employees across departments. For the specific exam dataset where max salaries are unique (30k, 40k, 50k), it produces the exact expected result: **Ravi, Nitin, Varun**.

IN and NOT IN Operators in SQL

Membership Evaluation, OR Equivalency & Subquery Application

16. The IN Operator

Set Membership

The IN operator allows you to easily test whether an expression matches any value in a list of literal values or a subquery result set. It serves as a concise shorthand for multiple OR conditions.

Verbose Equivalent (Multiple ORs)

```
SELECT * FROM Emp WHERE address =  
'Delhi' OR address = 'Chandigarh' OR address  
= 'Pune';
```

Clean Query with IN

```
SELECT * FROM Emp WHERE address IN (  
'Delhi', 'Chandigarh', 'Pune');
```

Mental Model for IN

Think: "Is this value included in the given set?"

- Chandigarh IN ('Delhi', 'Chandigarh', 'Pune') → **TRUE** (Included)
- Delhi IN ('Delhi', 'Chandigarh', 'Pune') → **TRUE** (Included)
- Bangalore IN ('Delhi', 'Chandigarh', 'Pune') → **FALSE** (Excluded)

17. The NOT IN Operator

Set Exclusion

The NOT IN operator performs the exact inverse operation. It selects records whose column value is **not found** within the provided set.

Query Example with NOT IN

```
SELECT * FROM Emp WHERE address NOT  
IN ( 'Delhi', 'Chandigarh', 'Pune' );
```

Evaluation Flow

City	IN Set?	NOT IN Result
Chandigarh	Yes	Rejected
Delhi / Pune	Yes	Rejected
Bangalore	No	Selected

18. Key Operator Comparisons & Subquery Integration

Exam Essentials

Operator	Logical Meaning	Example Condition
IN	Value matches any element in list (OR)	WHERE address IN ('Delhi', 'Pune')
NOT IN	Value matches no element in list (AND NOT)	WHERE address NOT IN ('Delhi', 'Pune')

Single Value Comparison (=)

Use when comparing against exactly one value.

```
WHERE salary = 50000
```

Multiple Value Comparison (IN)

Use when comparing against a list or multi-row subquery.

```
WHERE salary IN (30000, 40000,  
50000)
```